ASAP_APPS · RUST BACKEND · SUPABASE

ASAP Apps ↔ Rust Sync

Where the customer app, rider app, Supabase database and Rust API stand after the security lockdown, the signup reconciliation and the move to ride offers. It covers what each side has done and what still has to change before the flows work end to end.

As of 7 Oct 2026 App changes: uncommitted on `main`

Rust status: taken from the Rust agent's report, which has been type-checked but not run

Done | App change | Rust change | DB migration | Breaks a flow | You / ops

Fix first: mismatches between the apps and the new Rust code

The Rust agent changed the ride-request contract and the apps haven't caught up. Items marked Breaks a flow will stop a customer order from reaching a rider.

1. `order_ref` is required, but app orders don't have one

Breaks a flow | DB | App

Rust's `POST /riders/ride-request` now requires `order_ref` and looks it up in `app_delivery_orders`. That table has no `order_ref` column. Only `back_ride_request` has one, and no migration in this repo adds it. The customer app sends `order_ref: null` (`RiderAwaitingModal.tsx`), so every app order gets a 404.

- **DB** Migration: `alter table app_delivery_orders add column order_ref uuid not null default gen_random_uuid() unique;`. The default backfills existing rows.
- **Rust** Return `order_ref` from `/payments/korapay/initialize`, or let ride-request also find the order by `payment_reference`. Update `schema.rs`.
- **App** Read `order_ref` from the order row (the modal already loads it by `payment_reference`) and send it in `requestRide`.
- **Zazu** Every Zazu order needs its own `app_delivery_orders` row with this `order_ref` before it calls ride-request.

2. The waiting screen treats Rust's immediate 200 as "no riders"

Breaks a flow | App

The new ride-request replies straight away and searches in the background. `RiderAwaitingModal.tsx` still assumes the reply comes after the search has ended. If the row has no driver yet, it calls `noRiders()`. The customer would see "No riders found" right after paying.

- **App** On an ok reply, keep waiting. Only the row's `driver_id`, `status = 'no_driver'` or the timeout ends the wait. Raise `SEARCH_TIMEOUT_MS` above Rust's 3-minute search, for example to 3.5 minutes, so the app doesn't give up at the same moment Rust does.

3. "Try again" after `no_driver` may not restart the search

Breaks a flow · Rust · App

"Try again" calls ride-request again with the same payment. Rust says a retry with the same `order_ref` reuses the stored request and won't start a second search. That's correct while a search is running. Once the order is `no_driver`, though, the customer needs a new search.

Pick one (a) Ride-request resets a `no_driver` order to `pending` and searches again. Or (b) the app's "Try again" calls `POST /riders/assign-driver {order_ref}`, which Rust now offers for exactly this. Option (a) keeps the app simpler.

4. Assigned-mode offers never show in the rider app

Breaks a flow · App · DB

In assigned mode (Zazu by default), Rust inserts the offer as `status = 'accepted'` with `responded_at` empty, and takes it back if the rider doesn't tap Start within 60 s. `useRideOfferStore.ts` only shows offers with `status === 'pending'`, so these are dropped and every Zazu order times out. `get_offer_details` also doesn't return `responded_at`.

DB Recreate `get_offer_details` with `responded_at` in the result. Change `notify_ride_offer()` to set the push title with `case when new.status = 'accepted' then 'New delivery assigned' else 'New delivery request' end`.

App Show `accepted` offers that have no `responded_at`. `RideOfferModal` shows a Start-only card for them, which calls `/drivers/driver-response {offer_id, accepted: true}`.

5. Driver location on accept is sent but ignored

Rust

Offer accepts now send `{offer_id, accepted, driver_lat, driver_long}`, and the coordinates are null when unavailable. Rust's `driver-response` reads only `offer_id` and `accepted`. Nothing sets `driver_initial_lat/long` any more, so the customer's map has no starting point for the rider.

Rust Add `driver_lat: Option<f64>, driver_long: Option<f64>` to the payload. In the accept transaction, write them to `driver_initial_lat/long`. If they're null, fall back to `back_drivers` location.

6. `driver_arrived` status: nobody sets it yet

DB Rust App

Rust wants `driver_arrived` added to the enum and will count it as an active order. No app screen or Rust endpoint moves an order into it.

DB `alter type delivery_status add value if not exists 'driver_arrived' after 'arriving_pickup';`. Then uncomment the arrived-at-pickup branch in `notify_order_status_change()` under the new name, so the customer gets "Rider at pickup".

Rust Add an endpoint such as `POST /drivers/arrived-pickup {order_ref}`, using the token's driver id, that sets the status. Add the value to the Diesel enum.

App Add an "I've arrived" button to the rider app, plus status handling in both apps (status chips, tracking screen).

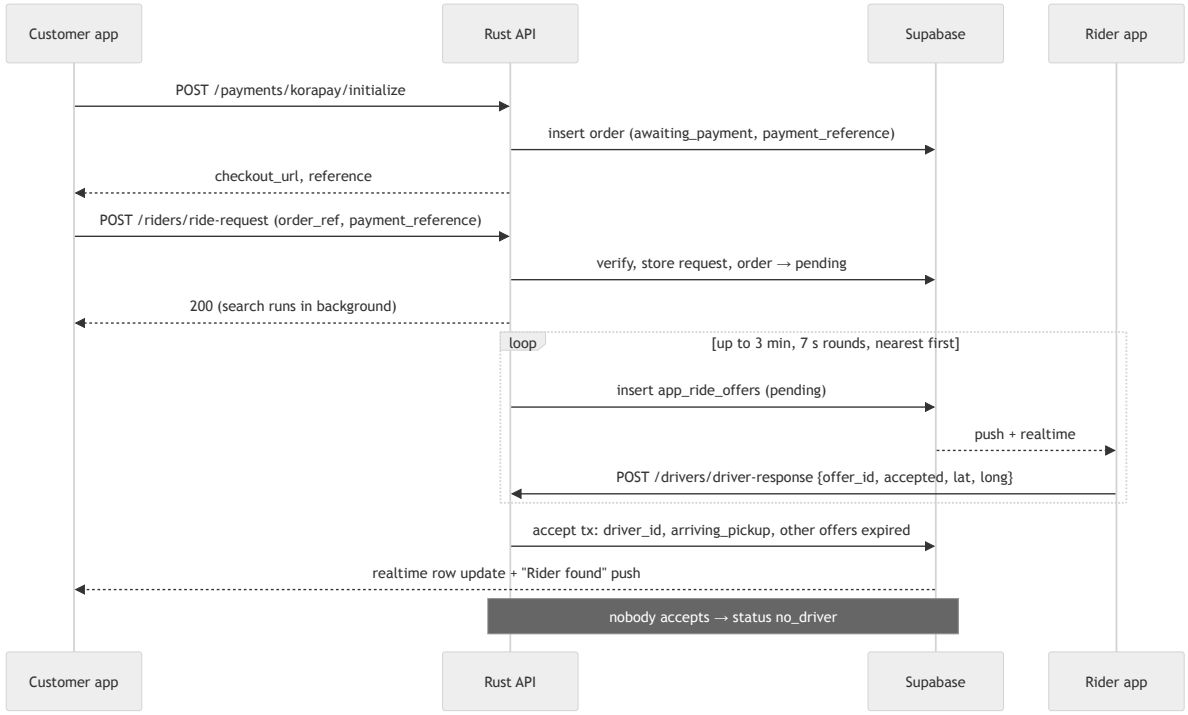
7. Code-confirm endpoints still take `driver_id` from the body

Rust

`confirm-dropoff` and `confirm-return` send `driver_id` in the body. All three confirm endpoints, `confirm-pickup` included, send the order's `order_code` as `order_ref`. Rust now takes the driver id from the token in `driver-response`.

Rust Take the driver id from the token in every `/drivers/*` handler and ignore the body value. Look orders up by `order_code`, or switch everything to the new `order_ref` column. The app will change too if you choose the column.

How an app order flows now



Order status values

`awaiting_payment` → `pending` → `arriving_pickup` → `driver_arrived (to add)` → `in_transit` → `delivered`

Side exits: `no_driver` `cancelled`

Offer statuses (`app_ride_offers.status`): `pending` , `accepted` , `rejected` , `expired` . At most one pending offer per order. Dispatch mode follows the order's `source` : anything listed in `ASSIGNED_DISPATCH_SOURCES` (default `zazu`) is assigned, everything else gets offers.

API contract: every Rust endpoint the apps call

ENDPOINT	CALLER	APP SENDS	APP EXPECTS	STATUS
`POST /riders/create-rider`	Customer signup; also retried on session load by `ensureCustomerAccount`	`{name, email, phone}`	2xx. Must be idempotent.	Rust Confirm the path. Take `rider_id` from the token, generate the pubkey, `on_conflict do_nothing` .

ENDPOINT	CALLER	APP SENDS	APP EXPECTS	STATUS
`POST /drivers/create-driver`	Rider signup	`{name, email, phone, vehicle_type: "Bike"|"EV", vehicle, license_number, account_number: null, bank_code: null}`	2xx. The app lets the user retry if it fails.	Rust Confirm the path. Use the token's id, generate the pubkey, be idempotent, and require `vehicle` and `license_number` when the type is EV.
`POST /payments/korapay/initialize`	Customer checkout	The ride request minus `payment_reference`, plus `redirect_url`	`{checkout_url, reference}`, and `order_ref` too (issue 1)	Rust Last noted as not built. Confirm.
`POST /payments/korapay/cancel`	Abandoned checkout	`{reference}`	`{status: "paid"|"cancelled"}`	Rust Confirm.
`POST /riders/ride-request`	RiderAwaitingModal	CreateRideRequest with `order_ref: null` (wrong now)	Body ignored	Rust done App + DB issues 1–3
`POST /drivers/driver-response`	RideOfferModal	`{offer_id, accepted, driver_lat, driver_long}`	2xx; 409 means expired or taken (the app closes the card); 404 means not theirs	Rust done Rust read lat/long (issue 5)
`POST /riders/assign-driver`	Not called (was GET; the rider list is hidden)	Now `{order_ref}`	n/a	Done Possible use for "Try again" (issue 3)
`POST /drivers/update-status`	Rider online toggle	`{status: "available"|"offline"}`	2xx, body ignored	Rust Must accept `offline` even while `busy`. The app already sends it.
`POST /drivers/confirm-pickup`	Customer enters the pickup code	`{order_ref: order_code, pickup_code}`	2xx; error text shown on failure	Rust Sets `in_transit` and recalculates driver status.
`POST /drivers/confirm-dropoff`	Rider enters the dropoff code	`{order_ref: order_code, driver_id, dropoff_code}`	2xx; error text shown on failure	Rust Use the token id (issue 7). Release escrow and recalculate driver status.
`POST /drivers/confirm-return`	Rider at the return point	`{order_ref, driver_id, return_code}`	2xx; error text on a bad code	Rust Not built. Set `app_returned.status = 'returned'`.
`POST /orders/cancel`	Customer cancels a paid order	`{payment_reference}`	`{refund_amount}` in naira; 0 = none	Rust Confirm. Post-pickup cancels also insert `app_returned` with a 6-digit return code.
`POST /riders/ride-quote`	Map / booking sheet	`{pick_up, drop_off}`	`RideQuote[]`	Done

ENDPOINT	CALLER	APP SENDS	APP EXPECTS	STATUS
`POST /drivers/update-bank-details`, `GET /trips/get-trip/:code`, `POST /matching/process-geolocation/:id`	Rider app	Unchanged	Unchanged	**Check** Not touched in this round. `process-geolocation` puts the user id in the path, so it should take the id from the token.

Removed in Rust: `/riders/wait-driver-response`, `notify_driver_handler` and its route, `DRIVER_NOTIFY_CHANNELS`. No app calls any of them.

Database migrations

FILE	WHAT IT DOES	STATE
20260926120000	`delete_own_account()` for in-app account deletion; refuses mid-delivery	Pushed
20260926130000	Push tokens and `send_push` via pg_net; customer status pushes; `app_ride_offers`, its push trigger and `get_offer_details`; order `source` and contact phones; realtime publication	Pushed
20260927120000	`no_driver` enum value	Pushed
20260927121000	`get_offer_details` returns the item photo	Pushed
20260927122000	`append_waypoints` fixed; only the assigned driver can write	Pushed
20260927123000	Chat photos: `image_url` and the private `chat_images` bucket	Pushed
20260929120000	`awaiting_payment` and `cancelled` states, cancel columns	Pushed
20260929120500	Push to the rider when their order is cancelled	Pushed
20260929121000	Chat locked to the two parties on an order	Pushed
20260929122000	`app_returned` table; `return_code` hidden behind `get_return_code`	Pushed
20260929130000	`back_drivers.tier` (hp 70% / or 84%)	Pushed
20261002120000	Locks down `app_custom_users` and `back_drivers`: anon gets nothing; bank, email and key columns are hidden; `shares_order_with()`	Pushed & verified
20261003120000	Locks down `back_custom_users`, `back_ride_request` and `back_trips`	**Run db push** Not pushed as of 3 Oct

Still to write

- DB `app_delivery_orders.order_ref uuid unique` (issue 1)
- DB `driver_arrived` enum value, plus its customer push branch (issue 6)
- DB `notify_ride_offer()` title for assigned offers; `get_offer_details` returns `responded_at` (issue 4)

- DB Capture what was run by hand in the SQL editor so a fresh database matches: the customer-only `get_return_code`, the new `notify_order_status_change` text and `driver_rides_completed`
- DB Later (Rust TODO 5): revoke client update and delete on `app_delivery_orders` once Rust owns every order write. The customer app still deletes orders directly (`deleteDeliveryByOrderCode` in `useCustomerDeliveriesStore.tsx`), so that needs a Rust endpoint first.

App changes this round (uncommitted)

Customer app

- Signup calls `createCustomerAccount` (Rust `create-rider`) after Supabase signup.
- `ensureCustomerAccount` runs on every session load and creates the Rust account if it's missing.
- `supabase_types.ts` : the stale `back_riders` entry is renamed to `back_custom_users` .

Rider app

- Signup collects vehicle type (Bike or EV), vehicle and licence; EV requires both. It then calls `createDriverAccount` , and if the Rust step fails the rider can tap again without signing up twice.
- Home: orders come only as offers. The `/riders/assign-driver` list is no longer fetched and its dropdown is hidden behind `SHOW_AVAILABLE_ORDERS_LIST = false` .
- Offer accept sends `driver_lat/long` from the last known position (under 60 s old), or a quick fresh fix.

Type-checking passes for the rider app. For the customer app, the only remaining error comes from `components/ui/collapsible.tsx`, which was already broken before this round. Neither app has been run on a device with these changes.

Rust: done vs. still needed

Done per the Rust report

- Ride-request replies straight away; it validates, stores the request and sets `pending` in one transaction. Retries are idempotent on `order_ref`.
- Background search: nearest drivers first, 10 per round, 7 s between rounds, 3-minute limit, then `no_driver`. It skips drivers who declined, let the offer expire or hold a live offer elsewhere. Expiry uses the DB clock.
- `driver-response` takes the driver id from the token and runs the accept transaction (claims the order only if it's unclaimed and pending, expires other offers, refreshes driver status).
- Assigned mode for `ASSIGNED_DISPATCH_SOURCES`, with a 60 s Start window before the order is taken back.
- Driver status is derived from a live active-order count against the limit (Bike 2, others 1). It never touches offline drivers.
- `simulate_driver.sh` polls `app_ride_offers` with the driver's JWT.

Needed

- Issues 1, 3, 5, 6 and 7 above.
- Regenerate `schema.rs` from the live DB. The current one is stale: no `app_` prefixes, wrong `delivery_orders` columns, and the enum is missing `awaiting_payment`, `no_driver` and `cancelled`, with `driver_arrived` to come.
- Ride-request should take the customer id from the token, not the body's `rider_id`.
- Return code generation and `confirm-return` (returns flow).
- Escrow release on dropoff and refunds per the cancel rules: full refund before a rider is assigned, none after.
- Sweep stale `awaiting_payment` orders after checking Korapay.
- Rust holds the `service_role` key; the apps never do.

Ops checklist

- **You** `supabase db push` for `20261003120000`.
- **You** Rebuild both apps with the real anon key, which is now in `.env`.
- **You** Rotate the `service_role` key if any earlier build shipped it in `SUPABASE_ANON_KEY`, then update Rust's env.
- **You** Add `WEBSITE_URL` to `.env` and EAS.
- **You** Zazu: read results from the order row (ride-request no longer waits for a driver), and create the order row with `order_ref` first.

Unused code for co-dev review

Nothing has been deleted. These are unused after this round and await sign-off.

APP	CODE	WHY IT'S UNUSED
Rider	`fetchAvailableOrders` (live and commented copies), `useRiderOrdersStore` and its type in `my_types.ts`	Offers replaced the order list

APP	CODE	WHY IT'S UNUSED
Rider	`AvailableOrdersDropdown.tsx`, `OrderSummary.tsx`, `acceptDeliveryOrder` (live and commented)	The list accept path; it also uses a GET on what is now a POST endpoint
Rider	`verifyDeliveryCode` (`/trips/update-trip`) and its commented predecessor	Replaced by `confirm-dropoff`
Customer	`getPendingOrdersWithRider`	Calls the old GET `/riders/assign-driver` ; the rider card uses `getOrderRiderInfo`
Customer	`hasDriverAcceptedDelivery` (`supabase-utils.ts`), commented `upsertDeliveryOrder`	Client-side order writes; Rust owns these now
Customer	`components/ui/collapsible.tsx`	Imports deleted themed components; the only remaining type-check error